

Web Server Go: Membangun REST API Pertama Anda Menggunakan net/http dan Gin Framework

Durasi: 3 Jam (180 Menit) **Level:** Intermediate (Menengah)

Sebagai Backend Engineer Go, salah satu tugas paling utama adalah menyediakan layanan API yang berkinerja tinggi. Di ekosistem Go, kita dapat membangun web server menggunakan pustaka bawaan `net/http`. Namun, untuk kebutuhan produksi dengan perutean (*routing*) yang kompleks, manajemen request/response yang bersih, serta penanganan *middleware*, komunitas Go banyak mengadopsi framework populer seperti **Gin**.

Modul ini akan membedah cara membangun web server dasar dengan `net/http`, mengidentifikasi batasannya, berkenalan dengan Gin Web Framework, memahami perutean dan pengelompokan API (*Route Grouping*), serta menguasai teknik memproses input (*Request Binding*) dan mengirimkan output (*Response Rendering*).

Sesi 1: Dasar Web Server dengan `net/http` (45 Menit)

Go menyediakan pustaka bawaan yang sangat tangguh bernama `net/http`. Banyak web server skala besar dibangun langsung menggunakan pustaka ini tanpa framework tambahan.

1. Membuat HTTP Server Sederhana

Untuk mendirikan web server, kita memerlukan dua hal dasar:

- **Handler:** Fungsi yang memproses HTTP Request masuk dan meresponsnya melalui HTTP Response.
- **Multiplexer (Mux):** Pengarah rute (*router*) yang mencocokkan URL request dengan handler yang tepat.

```
package main

import (
    "fmt"
    "net/http"
)

func main() {
    // 1. Definisikan Router / Multiplexer bawaan
```

```

mux := http.NewServeMux()

// 2. Hubungkan pola URL dengan fungsi Handler
mux.HandleFunc("/hello", func(w http.ResponseWriter, r *http.Request) {
    // Menulis HTTP Response Body
    fmt.Fprintf(w, "Halo, selamat datang di Web Server Go!")
})

fmt.Println("Server berjalan di http://localhost:8080...")
// 3. Jalankan server pada port 8080
if err := http.ListenAndServe(":8080", mux); err != nil {
    panic(err)
}
}

```

2. Memahami `http.ResponseWriter` dan `*http.Request`

Fungsi handler wajib memiliki tanda tangan (*signature*) berikut: `func(w http.ResponseWriter, r *http.Request)`

- `http.ResponseWriter` : Interface untuk mengirimkan respons balik ke klien (menulis header, status code, dan body).
- `*http.Request` : Pointer struct yang berisi informasi lengkap dari HTTP Request klien (URL, Header, Method, Body, Query Params).

3. Batasan Pustaka Bawaan `net/http`

Meskipun sangat cepat dan efisien, `net/http` memiliki beberapa batasan bawaan yang menyulitkan pembuatan REST API modern:

1. **Tidak Mendukung Path/URL Parameter Dinamis secara Out-of-the-box**: Mencocokkan rute seperti `/api/products/:id` membutuhkan parser manual yang rumit.
2. **Pengecekan HTTP Method Manual**: Router bawaan tidak memisahkan method `GET`, `POST`, `PUT`, `DELETE` secara otomatis. Anda harus menyaringnya sendiri menggunakan percabangan `if r.Method == http.MethodPost`.
3. **Parsing JSON Manual**: Membaca request body JSON dan mengubahnya menjadi struct Go membutuhkan proses pembacaan stream bytes (`io.ReadAll`) dan unmarshal manual berulang kali.

NOTE

Karena batasan-batasan tersebut, menggunakan HTTP framework seperti **Gin** sangat direkomendasikan untuk meningkatkan produktivitas tim, mempercepat proses pengembangan, dan menjaga kode tetap bersih (*clean code*).

Sesi 2: Pengenalan Gin Web Framework & Routing (45 Menit)

Gin adalah framework HTTP web berkinerja tinggi yang ditulis dalam bahasa Go. Gin menggunakan versi kustom dari router **HttpRouter** yang berbasis struktur data **Radix Tree** untuk perutean super cepat dengan konsumsi memori minimal.

1. Inisialisasi Proyek Gin

Untuk menggunakan Gin, pertama pastikan modul Go Anda telah terinisialisasi, lalu pasang pustaka Gin:

```
go get -u github.com/gin-gonic/gin
```

2. Membuat Server Pertama dengan Gin

```
package main

import "github.com/gin-gonic/gin"

func main() {
    // Inisialisasi engine default Gin (termasuk Logger & Recovery Middleware bawaan)
    r := gin.Default()

    // Routing sederhana
    r.GET("/ping", func(c *gin.Context) {
        c.JSON(200, gin.H{
            "message": "pong",
        })
    })

    // Jalankan server di port 8080
    r.Run(":8080")
}
```

- `gin.Default()` : Membuat router Engine dengan Logger (untuk mencetak log request di console) dan Recovery middleware (menangkap panic agar server tidak crash dan merespons dengan status 500).
- `gin.Context` : Ini adalah jantung dari Gin. Konteks ini menampung informasi request, menulis response, serta memfasilitasi passing data antar-middleware.

3. HTTP Methods & Route Grouping

Gin mempermudah deklarasi rute berdasarkan HTTP Methods dan memungkinkan pengelompokan rute (*Route Grouping*) untuk merapikan prefix URL:

```
package main

import "github.com/gin-gonic/gin"

func main() {
    r := gin.Default()

    // Pengelompokan Rute untuk Versi API
    v1 := r.Group("/api/v1")
    {
        // HTTP GET: Membaca Data
        v1.GET("/products", func(c *gin.Context) {
            c.JSON(200, gin.H{"data": "Semua produk"})
        })

        // HTTP POST: Membuat Data Baru
        v1.POST("/products", func(c *gin.Context) {
            c.JSON(201, gin.H{"message": "Produk berhasil dibuat"})
        })

        // HTTP PUT: Mengubah Data Secara Keseluruhan
        v1.PUT("/products", func(c *gin.Context) {
            c.JSON(200, gin.H{"message": "Produk berhasil diperbarui"})
        })

        // HTTP DELETE: Menghapus Data
        v1.DELETE("/products", func(c *gin.Context) {
            c.JSON(200, gin.H{"message": "Produk berhasil dihapus"})
        })
    }

    r.Run(":8080")
}
```

Sesi 3: Menangani Request & Response di Gin (45 Menit)

Sebagai Backend Engineer, Anda wajib memahami 3 metode utama pengiriman data dari klien: **Query Parameters**, **Path Parameters**, dan **Request Body JSON**.

1. Mengambil Query Parameters (`c.Query`)

Query parameters berada di ujung URL setelah tanda tanya (`?key=value`). Biasanya digunakan untuk filtrasi, pencarian, atau paginasi.

- `c.Query("key")` : Mengambil nilai parameter. Mengembalikan string kosong jika kunci tidak ada.
- `c.DefaultQuery("key", "default_val")` : Mengembalikan nilai default jika kunci tidak disediakan klien.

```
r.GET("/search", func(c *gin.Context) {
    // http://localhost:8080/search?keyword=sepatu&page=2
    keyword := c.Query("keyword")
    page := c.DefaultQuery("page", "1") // Default halaman ke-1

    c.JSON(200, gin.H{
        "keyword": keyword,
        "page":    page,
    })
})
```

2. Mengambil Path/URL Parameters (`c.Param`)

Path parameters adalah variabel dinamis yang disematkan langsung di dalam rute URL (contoh: `/products/:id`).

- Menggunakan tanda titik dua (`:`) pada pola rute di Gin.

```
r.GET("/products/:id", func(c *gin.Context) {
    // http://localhost:8080/products/45
    id := c.Param("id") // id bernilai "45"

    c.JSON(200, gin.H{
        "product_id": id,
    })
})
```

3. Binding Request Body JSON (`c.ShouldBindJSON`)

Untuk menangkap data payload JSON yang dikirim melalui method POST atau PUT, Gin menyediakan fitur *Binding*. Nilai JSON akan dipetakan langsung ke struct Go menggunakan tag struct `json:"..."`. Kita juga bisa memvalidasi input secara otomatis menggunakan tag `binding:"required"`.

```
type ProductRequest struct {
    Name string `json:"name" binding:"required"`
```

```

    Price float64 `json:"price" binding:"required,gt=0"` // Harus diisi dan > 0
    Stock int      `json:"stock" binding:"required,min=0"` // Harus diisi dan ≥ 0
}

r.POST("/products", func(c *gin.Context) {
    var req ProductRequest

    // Validasi dan Binding JSON ke struct req
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{
            "error": err.Error(), -- Mengembalikan pesan kesalahan
validasi
        })
        return
    }

    // Jika validasi sukses, lakukan sesuatu dengan data req
    c.JSON(http.StatusCreated, gin.H{
        "message": "Produk valid!",
        "data":    req,
    })
})

```

WARNING

Validation Error Output: Gunakan method `c.ShouldBindJSON` daripada `c.BindJSON`. Bedanya, `ShouldBindJSON` tidak langsung menulis header 400 otomatis ketika binding gagal, sehingga Anda dapat memformat respons kesalahan dengan bentuk JSON kustom Anda sendiri.

Sesi 4: Praktikum Mandiri: REST API Produk E-Commerce Sederhana (45 Menit)

Mari kita gabungkan seluruh konsep di atas untuk membuat REST API e-commerce sederhana untuk manajemen produk secara *in-memory* (menggunakan slice global sebagai pengganti database).

Implementasi Kode (`main.go`)

```

package main

import (
    "net/http"
    "strconv"

```

```

        "github.com/gin-gonic/gin"
    )

    // 1. Struct Model Data & Validation
    type Product struct {
        ID      int      `json:"id"`
        Name    string   `json:"name" binding:"required"`
        Price   float64  `json:"price" binding:"required,gt=0"`
        Stock   int      `json:"stock" binding:"required,min=0"`
    }

    // Data in-memory database
    var products = []Product{
        {ID: 1, Name: "Smartphone iPhone 15 Pro", Price: 21000000.00, Stock: 10},
        {ID: 2, Name: "TWS Sony WF-1000XM5", Price: 3500000.00, Stock: 25},
    }
    var nextID = 3

    func main() {
        r := gin.Default()

        // Route Grouping
        api := r.Group("/api")
        {
            // GET /api/products - Mengambil semua produk dengan filter nama
            optional

            api.GET("/products", func(c *gin.Context) {
                search := c.Query("search")
                if search == "" {
                    c.JSON(http.StatusOK, gin.H{"data": products})
                    return
                }

                // Filter berdasarkan kata kunci nama
                var filtered []Product
                for _, p := range products {
                    if containsIgnoreCase(p.Name, search) {
                        filtered = append(filtered, p)
                    }
                }
                c.JSON(http.StatusOK, gin.H{"data": filtered})
            })

            // GET /api/products/:id - Mengambil produk spesifik
            api.GET("/products/:id", func(c *gin.Context) {
                idStr := c.Param("id")
                id, err := strconv.Atoi(idStr)
                if err != nil {
                    c.JSON(http.StatusBadRequest, gin.H{"error": "ID
produk harus berupa angka"})
                    return
                }
            })
        }
    }

```

```

        for _, p := range products {
            if p.ID == id {
                c.JSON(http.StatusOK, gin.H{"data": p})
                return
            }
        }

        c.JSON(http.StatusNotFound, gin.H{"error": "Produk tidak
ditemukan"})
    })

    // POST /api/products - Membuat produk baru
    api.POST("/products", func(c *gin.Context) {
        var newProduct Product
        if err := c.ShouldBindJSON(&newProduct); err != nil {
            c.JSON(http.StatusBadRequest, gin.H{"error":
err.Error()})

            return
        }

        newProduct.ID = nextID
        nextID++
        products = append(products, newProduct)

        c.JSON(http.StatusCreated, gin.H{
            "message": "Produk berhasil dibuat!",
            "data":    newProduct,
        })
    })

    // DELETE /api/products/:id - Menghapus produk
    api.DELETE("/products/:id", func(c *gin.Context) {
        idStr := c.Param("id")
        id, err := strconv.Atoi(idStr)
        if err != nil {
            c.JSON(http.StatusBadRequest, gin.H{"error": "ID
produk harus berupa angka"})

            return
        }

        for idx, p := range products {
            if p.ID == id {
                // Hapus elemen dari slice products
                products = append(products[:idx],
products[idx+1:]...)

                c.JSON(http.StatusOK, gin.H{"message": "Produk
berhasil dihapus"})

                return
            }
        }

        c.JSON(http.StatusNotFound, gin.H{"error": "Produk tidak

```



```

    ditemukan"}})
    })
}

r.Run(":8080")
}

// Fungsi helper sederhana untuk filter case-insensitive search
func containsIgnoreCase(src, search string) bool {
    // Konversi manual demi kesederhanaan helper
    return len(src) ≥ len(search) && (contains(stringsToLower(src),
stringsToLower(search)))
}

func stringsToLower(s string) string {
    var result []rune
    for _, r := range s {
        if r ≥ 'A' && r ≤ 'Z' {
            result = append(result, r+32)
        } else {
            result = append(result, r)
        }
    }
    return string(result)
}

func contains(s, substr string) bool {
    if len(substr) = 0 {
        return true
    }
    for i := 0; i ≤ len(s)-len(substr); i++ {
        if s[i:i+len(substr)] = substr {
            return true
        }
    }
    return false
}

```

Sesi 5: Soal Latihan Praktik E-Commerce (Mandiri)

Selesaikan dua latihan pemrograman web REST API di bawah ini untuk memperkuat pemahaman Anda mengenai framework Gin, validasi binding, dan perutean.

Latihan 1: REST API Sistem Pengelolaan Kategori Barang dengan Validasi Custom

Deskripsi Soal: Buatlah REST API lengkap menggunakan Gin untuk manajemen kategori barang e-commerce secara in-memory.

Kriteria Implementasi:

1. Buat struct model `Category` yang berisi:
 - `ID` (int)
 - `Name` (string) - Wajib diisi, minimal 3 karakter.
 - `Code` (string) - Wajib diisi, tidak boleh mengandung spasi (gunakan validasi Gin jika bisa, atau lakukan pengecekan manual di handler).
 2. Sediakan rute-rute endpoint berikut:
 - `POST /api/categories` : Menambahkan kategori baru (lakukan validasi struct binding).
 - `GET /api/categories` : Menampilkan seluruh daftar kategori.
 - `GET /api/categories/:code` : Mengambil satu kategori berdasarkan kode uniknya. Jika kode tidak cocok, kirim response 404 dengan pesan yang sesuai.
 3. Uji API menggunakan REST Client (seperti Postman, Insomnia, atau Thunder Client) dan cetak hasilnya dengan rapi.
-

Latihan 2: REST API Pembayaran Transaksi (Checkout) dengan Validasi Struct Tingkat Lanjut

Deskripsi Soal: Platform e-commerce membutuhkan REST API checkout order yang dapat menerima item keranjang belanja dari klien, menghitung total belanjaan secara dinamis, dan melakukan validasi kelayakan data.

Kriteria Implementasi:

1. Definisikan struct request input berikut:
 - `CartItem` :
 - `ProductID` (int) - Wajib diisi.
 - `Quantity` (int) - Wajib diisi, minimal bernilai 1.
 - `CheckoutRequest` :
 - `CustomerName` (string) - Wajib diisi.
 - `Items` (`[]CartItem`) - Minimal harus ada 1 item yang dibeli (gunakan tag `binding:"required,dive"` untuk memvalidasi elemen di dalam slice).
 - `PaymentMethod` (string) - Wajib diisi, hanya boleh menerima nilai `"CASH"`, `"E-WALLET"`, atau `"BANK_TRANSFER"`.
2. Buat endpoint `POST /api/checkout`. Di dalam handlernya:
 - Lakukan binding JSON request ke struct `CheckoutRequest`.
 - Jika binding gagal, kembalikan status 400 Bad Request beserta pesan kesalahannya.

- Jika validasi lolos, hitung nominal total tagihan pembayaran secara virtual (misal setiap `ProductID` diasumsikan memiliki harga tetap Rp 100.000).
 - Kembalikan response status 201 Created yang menyertakan informasi detail pemesan, jumlah item, total pembayaran, dan status transaksi `'PENDING_PAYMENT'`.
-

Sesi 6: Ujian Project Akhir – Mini HRIS (Human Resource Information System)

Deskripsi Tugas Project

Sebagai bentuk evaluasi akhir, Anda diminta untuk membangun sebuah backend **Mini HRIS (Human Resource Information System)** menggunakan Go, framework **Gin Gonic**, dan database **SQLite** (menggunakan driver SQLite bawaan Go atau melalui GORM). Sistem ini wajib diorganisasikan menggunakan arsitektur folder yang rapi dan menangani operasi CRUD relasional yang kompleks.

1. Desain Skema Database (Minimal 6 Tabel)

Database harus dirancang dengan relasi kunci asing (*Foreign Key*) yang benar. Definisikan minimal 6 entitas tabel berikut:

1. Departments (Departemen):

- `ID` (Primary Key, Auto Increment)
- `Name` (string) - Nama departemen (misal: IT, HR, Finance).
- `Code` (string) - Kode departemen unik (misal: DEPT-IT).

2. Positions (Jabatan):

- `ID` (Primary Key, Auto Increment)
- `Title` (string) - Nama jabatan (misal: Software Engineer, HR Specialist).
- `BaseSalary` (float64) - Gaji pokok bawaan untuk jabatan tersebut.

3. Employees (Karyawan):

- `ID` (Primary Key, Auto Increment)
- `NIK` (string) - Nomor Induk Karyawan unik.
- `FullName` (string) - Nama lengkap karyawan.
- `Email` (string) - Email karyawan unik (format email valid).
- `DepartmentID` (Foreign Key -> Departments)
- `PositionID` (Foreign Key -> Positions)
- `Status` (string) - Status karyawan (`"ACTIVE"` , `"SUSPENDED"` , `"TERMINATED"`).

4. Attendances (Kehadiran):

- `ID` (Primary Key, Auto Increment)
- `EmployeeID` (Foreign Key -> Employees)
- `Date` (string/Date) - Tanggal kehadiran (format `YYYY-MM-DD`).

- `CheckIn` (string) - Jam masuk (format `HH:MM`).
- `CheckOut` (string) - Jam pulang (format `HH:MM`).
- `Status` (string) - `"PRESENT"` , `"LATE"` , `"ABSENT"` .

5. Leaves (Pengajuan Cuti):

- `ID` (Primary Key, Auto Increment)
- `EmployeeID` (Foreign Key -> Employees)
- `StartDate` (string/Date) - Tanggal mulai cuti.
- `EndDate` (string/Date) - Tanggal selesai cuti.
- `Reason` (string) - Alasan cuti.
- `Status` (string) - Status persetujuan (`"PENDING"` , `"APPROVED"` , `"REJECTED"`).

6. Salaries (Slip Gaji/Payroll):

- `ID` (Primary Key, Auto Increment)
- `EmployeeID` (Foreign Key -> Employees)
- `Period` (string) - Bulan dan tahun periode gaji (format `YYYY-MM`).
- `BasicSalary` (float64) - Gaji pokok saat pembayaran.
- `Allowance` (float64) - Tunjangan kehadiran (misal: kehadiran tepat waktu mendapat bonus Rp 50.000 per hari).
- `Deductions` (float64) - Potongan (misal: setiap hari terlambat dipotong Rp 20.000, absen dipotong Rp 100.000).
- `NetSalary` (float64) - Gaji bersih akhir yang dihitung dinamis (`BasicSalary` + `Allowance` - `Deductions`).

2. Kriteria & Fitur Wajib REST API

Implementasikan endpoint API berikut dengan validasi binding JSON yang ketat:

A. Pengelolaan Data Master (CRUD Lengkap)

- **Departments:** CRUD (`POST /api/departments` , `GET /api/departments` , `PUT /api/departments/:id` , `DELETE /api/departments/:id`).
- **Positions:** CRUD (`POST /api/positions` , `GET /api/positions` , `PUT /api/positions/:id` , `DELETE /api/positions/:id`).

B. Manajemen Karyawan (Relasional)

- `POST /api/employees` : Menambahkan karyawan baru dengan memvalidasi keberadaan `DepartmentID` dan `PositionID` .
- `GET /api/employees` : Menampilkan semua daftar karyawan beserta data detail departemen dan jabatannya (gunakan SQL JOIN atau Preload GORM). Wajib dilengkapi dengan fitur pencarian nama karyawan (`?search=nama`) serta filter berdasarkan `department_id` , `position_id` , dan `status` (`ACTIVE` / `SUSPENDED` / `TERMINATED`).
- `PUT /api/employees/:id` : Memperbarui data karyawan termasuk mengubah departemen atau jabatannya.

C. Pencatatan Kehadiran & Pengajuan Cuti

- `POST /api/attendances` : Karyawan melakukan pencatatan jam masuk (`CheckIn`) dan keluar (`CheckOut`).
- `POST /api/leaves` : Karyawan mengajukan cuti (`PENDING`).
- `PATCH /api/leaves/:id/approve` : HR menyetujui (`APPROVED`) atau menolak (`REJECTED`) pengajuan cuti karyawan.

D. Proses Payroll Bulanan (Perhitungan Dinamis)

- `POST /api/salaries/calculate` : Endpoint untuk menghitung dan menyimpan rekap gaji bersih karyawan untuk periode tertentu. Di dalam proses ini:
 1. Ambil data gaji pokok berdasarkan jabatan karyawan.
 2. Hitung jumlah kehadiran dari tabel `Attendances` (untuk menentukan `Allowance` dan `Deductions`).
 3. Kurangi saldo jatah cuti jika ada cuti yang disetujui (`APPROVED`) di tabel `Leaves` .
 4. Simpan hasil akhir ke tabel `Salaries` .
 - `GET /api/salaries/period/:period` : Menampilkan daftar seluruh slip gaji karyawan pada bulan tertentu.
-

3. Syarat Teknis Pengumpulan

1. **Struktur Repositori**: Wajib memisahkan kode menjadi folder terstruktur, minimal membagi berkas menjadi:
 - `config/` (koneksi SQLite)
 - `models/` (definisi struct entitas)
 - `controllers/` (logika handler API)
 - `routes/` (registrasi perutean Gin)
 - `main.go` (titik masuk utama aplikasi)
2. **Validasi**: Gunakan tag validator Gin (misal `binding:"required,email"` , dll.) dan tangani error binding dengan mengembalikan response status 400 Bad Request.
3. **Database Seeding**: Sediakan data awal (*seeder*) minimal untuk data Departemen, Jabatan, dan beberapa Karyawan agar API langsung dapat diuji dengan mudah.